

Enums in C# with Examples

In this article, I am going to discuss **Enums in C#** with Examples. Please read our previous article where we discussed [Indexers in C#](#) with Examples. At the end of this article, you will understand what are Enums and when, and how to use Enums in C# with Examples.

Why do we need Enums in C#?

The Enums are Strongly Typed Name Constants. Let us understand Enums i.e. what do you mean by Strongly Typed Name Constants with an Example? Let us assume we have an Employee class with the Name and Gender Properties. Gender is an Integer Property and the values of this property represent the following.

1. 0 is an Unknown Gender
2. 1 is Male
3. 2 is Female

That means if the Value of the Gender Property is 1, then its meaning is Gender is Male, similarly, if the value is 2, then its meaning is Female, and for 0, the meaning is Gender is Unknow. For a better understanding, please have a look at the below example. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.Collections.Generic;
namespace EnumsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create a collection to store list of employees
            List<Employee> empList = new List<Employee>
            {
                new Employee() { Name = "Anurag", Gender = 0 },
                new Employee() { Name = "Pranaya", Gender = 1 },
                new Employee() { Name = "Priyanka", Gender = 2 },
                new Employee() { Name = "Sambit", Gender = 3 }
            };
            //Loop through Each Employees and Print the Name and Gender
            foreach (var emp in empList)
            {
                //To Print the Actual Gender of the Employee,
                //we need to call the GetGender Method by passing the Integer
                Value
                Console.WriteLine($"Name = {emp.Name} && Gender =
                {GetGender(emp.Gender)}");
            }
            Console.ReadLine();
        }
    }
}
```

```

Value //This Method is used to return the Actual Gender Based on the Integer
public static string GetGender(int gender)
{
    // The switch case here is less readable because of these integral
    switch (gender)
    {
        case 0:
            return "Unknown";
        case 1:
            return "Male";
        case 2:
            return "Female";
        default:
            return "Invalid Data for Gender";
    }
}

public class Employee
{
    public string Name { get; set; }
    // 0 - Unknown
    // 1 - Male
    // 2 - Female
    public int Gender { get; set; }
}

```

When we run the above program, we will get the following output as expected.

```

Name = Anurag && Gender = Unknown
Name = Pranaya && Gender = Male
Name = Priyanka && Gender = Female
Name = Sambit && Gender = Invalid Data for Gender

```

The downside of the above program is less readable as well as less maintainable. This is because it operates on integrals instead of using enums to get the gender. Now let's see how to replace these integral numbers with Enums to make the program more readable and maintainable. First, create an enum for the Gender as shown below. To create an Enum in C#, we need to use the enum keyword followed by the name, and within the curly braces ({}), we need to specify the named constants which are shown in the below image.

```
public enum Gender
{
    Unknown,
    Male,
    Female
}
```

As you can see in the above image, the Enum is created with three named constants i.e. Unknown, Male, and Female. Now, instead of the integer numbers, let us use the above Gender Enum Named Constants within the GetGender method to return the actual gender. So, with the Gender Enum, we need to modify the GetGender method as shown below.

```
//This Method is used to return the Actual Gender Based on the Enum Gender Value
public static string GetGender(int gender)
{
    // The switch case here is now more readable and maintainable because
    // of replacing the integral numbers with Gender Enum
    switch (gender)
    {
        case (int)Gender.Unknown:
            return "Unknown";
        case (int)Gender.Male:
            return "Male";
        case (int)Gender.Female:
            return "Female";
        default:
            return "Invalid Data for Gender";
    }
}
```

As you can see in the above code, now we are using Enums instead of integer integrals which makes the code more readable and maintainable. So, the complete example is given below. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.Collections.Generic;
namespace EnumsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create a collection to store list of employees
            List<Employee> empList = new List<Employee>
            {
```

```

        new Employee() { Name = "Anurag", Gender = 0 },
        new Employee() { Name = "Pranaya", Gender = 1 },
        new Employee() { Name = "Priyanka", Gender = 2 },
        new Employee() { Name = "Sambit", Gender = 3 }
    };
    //Loop through Each Employees and Print the Name and Gender
    foreach (var emp in empList)
    {
        //To Print the Actual Gender of the Employee,
        //we need to call the GetGender Method by passing the Integer
Value
        Console.WriteLine($"Name = {emp.Name} && Gender =
{GetGender(emp.Gender)}");
    }
    Console.ReadLine();
}
//This Method is used to return the Actual Gender Based on the Enum Gender
public static string GetGender(int gender)
{
    // The switch case here is now more readable and maintainable because
    // of replacing the integral numbers with Gender Enum
    switch (gender)
    {
        case (int)Gender.Unknown:
            return "Unknown";
        case (int)Gender.Male:
            return "Male";
        case (int)Gender.Female:
            return "Female";
        default:
            return "Invalid Data for Gender";
    }
}
}

public class Employee
{
    public string Name { get; set; }
    // 0 - Unknown
    // 1 - Male
    // 2 - Female
    public int Gender { get; set; }
}

public enum Gender
{
    Unknown,
    Male,
    Female
}
}

```

Now when you run the application you will get the output as expected as shown below.

```
Name = Anurag && Gender = Unknown  
Name = Pranaya && Gender = Male  
Name = Priyanka && Gender = Female  
Name = Sambit && Gender = Invalid Data for Gender
```

Note: If a program uses a set of Integral Numbers then consider them replacing with enums which makes the program more Readable and Maintainable. That means if you have a property that is going to hold integer numbers and for each integer number we have a string value, then consider replacing the integer numbers with Enums.

Points to Remember while working with Enums in C#:

1. The Enums are Enumerations.
2. Enums are **Strongly Typed Named Constants**. Hence, an Explicit Cast is needed to convert them from the enum type to an integral type and vice versa. Also, an enum of one type cannot be implicitly assigned to an enum of another type even though the underlying value of their members is the same.
3. The default underlying type of an enum is int.
4. The default value for the first element of the enum is ZERO and gets incremented by 1.
5. It is also possible to customize the underlying type and values of enums.
6. The Enums are value types.
7. Enum keyword (all small letters) is used to create the enumerations, whereas the Enum class, contains static GetValues() and GetNames() methods which can be used to list Enum underlying type values and Names.

What is the Default Type of Enum Members in C#?

The default underlying type of enum is int and the value starts at ZERO and is incremented by 1 for each enum member. For example, in the below Enum, the type of Unknown, Male, and Female, and the value for Unknown is 0, Male is 1, and Female is 2.

```
public enum Gender  
{  
    Unknown, //Type Int and Value = 0  
    Male, //Type Int and Value = 1  
    Female //Type Int and Value = 2  
}
```

Can we Change the Underlying type of Enum in C#?

Yes, we can change the Underlying type of Enum in C#. For that, our enum needs to be inherited from that specific type. For example, if we want the underlying type to be short, then our enum needs to be inherited from the short data type as shown in the below image. Now, the Gender enum underlying type is short and

the value starts from 1 and is incremented by 1 for each enum member. So, in this case, the value for Male is 2 and for Female, the value is 3.

```
public enum Gender : short
{
    Unknown = 1, //Type short and Value = 1
    Male, //Type short and Value = 2
    Female //Type short and Value = 3
}
```

Can we Assign Random Values to Enum Members in C#?

Yes, we can Assign Random Values to Enum Members in C#. The Enum values need not be in sequential order. Any valid underlying type value is allowed. For example, in the following Enum, we are specifying the Unknown value as 10, Male as 20, and Female as 25. Any random order is valid.

```
public enum Gender : short
{
    Unknown = 10, //Type short and Value = 10
    Male = 20, //Type short and Value = 20
    Female = 25 //Type short and Value = 25
}
```

The following enum will not be compiled. This is because the MaxValue = 32767 and MinValue = -32768 are allowed with short data types. And here, we are trying to store a value of 32768 which is beyond the range of short data type. So, the allowed values for the enum members depend on the underlying data type.

```
public enum Gender : short
{
    Unknown = 10,
    Male = 32768,
    Female = 25
}
```

 `struct System.Int32`

Represents a 32-bit signed integer.

Constant value '32768' cannot be converted to a 'short'

An explicit cast is needed to convert from an enum type to an integral type and vice versa. The following line will not compile. A compiler error will be generated stating: **Cannot implicitly convert type 'Gender' to 'int'. An explicit conversion exists (are you missing a cast?)**

```
int i = Gender.Male;
```

Again, the above line will also not compile. A slightly different compiler error will be generated stating: **The left-hand side of an assignment must be a variable, property, or indexer**
Gender female = 2;

An Enum of one type cannot be implicitly assigned to an Enum of another type:

The Enum of one type cannot be implicitly assigned to an enum of another type even though the underlying value of their members are the same. In such cases, an explicit cast is required. For a better understanding, please have a look at the below example. In the below example, I am showing how to explicitly cast an Enum of one type to another type in C#.

```
namespace EnumsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // This following line will not compile.
            // Cannot implicitly convert type 'Season' to 'Gender'.
            // An explicit conversion is required.
            // Gender gender = Season.Winter;
            // The following line compiles as we have an explicit cast
            Gender gender = (Gender)Season.Winter;
        }
    }
    //Gender Enum
    public enum Gender : int
    {
        Unknown = 1,
        Male = 2,
        Female = 3
    }
    //Season Enum
    public enum Season : int
    {
        Winter = 1,
        Spring = 2,
        Summer = 3
    }
}
```

Understanding GetValues() and GetNames() Methods of Enum Class:

As we already discussed, the **enum** keyword (all small letters) in C# is used to create enumerations whereas the Enum class in C# contains the static **GetValues()** and **GetNames()** methods which can be used to list the Enum underlying type values and Names.

1. **GetValues():** The GetValues static method of the Enum class is used to retrieve an array of the values of the constants in a specified enumeration.

2. **GetNames():** The GetNames static method of the Enum class is used to retrieve an array of the names of the constants in a specified enumeration.

While calling the above two methods, we need to pass the type of the enum. For a better understanding, please have a look at the following example. As you can see in the below code, while calling the **GetValues()** and **GetNames()** static methods of the Enum Class, we are passing the enum type i.e. `typeof(Gender)`. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace EnumsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //GetValues Method return an array of Values of Gender Enum
            //Values are nothing but Integer numbers i.e. 1, 2, and 3
            int[] EnumValues = (int[])Enum.GetValues(typeof(Gender));
            Console.WriteLine("Gender Enum Values");
            //Looping through the EnumValues array to Print all the Values
            foreach (int value in EnumValues)
            {
                Console.WriteLine(value);
            }
            Console.WriteLine();
            //GetNames Method return an array of Names of Gender Enum
            //Names are nothing but the string named constants i.e. Unknown, Male,
            Female

            string[] EnumNames = Enum.GetNames(typeof(Gender));
            Console.WriteLine("Gender Enum Names");
            //Looping through the EnumNames array to Print all the Names
            foreach (string Name in EnumNames)
            {
                Console.WriteLine(Name);
            }
            Console.ReadKey();
        }
    }
}
//Creating an Enum to Hold Gender-Named Constants
public enum Gender : int
{
    Unknown = 1,
    Male = 2,
    Female = 3
}
```

When we run the above code, it will give you the following output.

- 1
- 2
- 3

Unknown
Male
Female

The classes cannot be derived from enums. This is because Enums are treated as sealed classes and hence all rules that are applicable to sealed classes also apply to enums. Sealed means a class can not further take part in inheritance. For a better understanding, please have a look at the following example. Here, we have the enum Gender and class Employee. And class Employee trying to inherit from the Gender Enum and hence we are getting the compilation error.

```
//Creating an Enum to Hold Gender Named Constants
public enum Gender : int
{
    Unknown = 1,
    Male = 2,
    Female = 3
}

//Class trying to Inherit from Enum
public class Employee : Gender
{
}
```

 class EnumsDemo.Employee

'Employee': cannot derive from sealed type 'Gender'

Can an Enum Contain Duplicate Members in C#?

The Enum members should be unique. The members of an Enum cannot be duplicated in C#. If we try to create duplicate Enum members then we will get a compilation error. For a better understanding, please have a look at the following Enum. Here, we are creating the Enum Gender with the member Unknown two types and hence getting the compilation error.

```
//Creating an Enum to Hold Gender Named Constants
public enum Gender : int
{
    Unknown,
    Male,
    Female,
    Unknown
}

Gender.Unknown = 3
```

The type 'Gender' already contains a definition for 'Unknown'

Note: The Enums are like classes, so we cannot define two members with the same name.

In the next article, I am going to discuss [.NET Framework Architecture](#). Here, in this article, I try to explain Enums in C# with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.